Kafka Microservices with Spring Boot, K8s, Docker & AI

Build modern event-driven microservices with Kafka, Spring Boot 4.x,
PostgreSQL, Docker, Kubernetes, Automated Testing, and AI-Assisted Development



About Me

- Dilip
- Building Software’s since 2008
- Teaching Online since 2026

Why Learn Kafka Microservices?

- Modern applications need to be scalable, reliable, and event-driven.
- Apache Kafka is widely used for building systems that can handle:
- ✅ High-volume data streams
  ✅ Event-driven communication
  ✅ Reliable message processing
  ✅ Decoupled microservices
  ✅ Real-time application workflows
- Kafka is a powerful foundation for building modern event-driven applications.

Kafka Microservices Journey

- 🌱 **Kafka Fundamentals**
  - Understand Kafka basics, topics, producers, consumers, and event-driven communication.
- 📤 **Kafka Producer Development**
  - Build a Spring Boot Producer Microservice to publish events to Kafka.
- 📥 **Kafka Consumer Development**
  - Build a Spring Boot Consumer Microservice to consume and process events.
- 🛡️ **Producer & Consumer Resiliency**
  - Learn how to make both Kafka producers and consumers more reliable using safe producer configurations, acknowledgments, retries, recovery strategies, and error handling patterns.
- 🔐 **Kafka Transactions**
  - Explore transactional processing and exactly-once style workflows.
- 🐋 **Containerization with Docker**
  - Package Kafka microservices as Docker containers.
- ☸️ **Run in Kubernetes**
  - Deploy and run the complete application in Kubernetes.
- We will go from Kafka basics to building, testing, containerizing, and running production-style Kafka microservices in Kubernetes.

AI-Assisted Development & Agentic Engineering

- A key part of this course is learning how to use AI-assisted coding as part of the development workflow.
- We will use **AI** to help with:

✅ Understanding requirements

✅ Designing implementation plans

✅ Generating code

✅ Writing tests

✅ Refactoring code

✅ Improving developer productivity

- **Agentic Engineering** is the modern way of building applications where developers guide AI coding agents with:

✅ Clear requirements

✅ Strong context

✅ Engineering patterns

✅ Testing expectations

✅ Code review mindset

Hands-On Learning Approach

- This course follows a practical, step-by-step learning approach.
- We will:

  ✅ Start with the core concepts

  ✅ Build the application incrementally

  ✅ Use AI-assisted coding throughout the course

  ✅ Test each important feature

  ✅ Package the applications using Docker

  ✅ Deploy the applications to Kubernetes

- You will not just learn the theory — you will build the application step by step.

Course Goal

- By the end of this course, you will be able to build modern Kafka microservices using:

  ✅ Spring Boot

  ✅ Apache Kafka

  ✅ PostgreSQL

  ✅ Docker

  ✅ Kubernetes

  ✅ Automated Testing

  ✅ AI-Assisted Development

- You will understand how to move from local development to containerized applications and Kubernetes deployment.

🙏 Thank You for Joining This Course

Course Prerequisites

- To get the best learning experience from this course, it is helpful to have a basic understanding of:
  - ☕ Java programming fundamentals
  - 🍃 Spring Boot basics
  - 🌐 REST API development
  - 🗄️ Basic database concepts
  - 💻 Basic command-line usage
  - 🔀 Basic Git knowledge
- You do not need to be an expert in Kafka, Docker, Kubernetes, or AI-assisted development.
  - We will explore those topics step by step in the course.

Whats Covered?

- Introduction to Kafka and internals of Kafka
- Building Enterprise standard Kafka Clients using **Spring-Kafka/ SpringBoot**
- Resilient Kafka Client applications using **Error-Handling/Retry/Recovery**
- Writing Unit/Integration tests using **JUnit**

Targeted Audience

- Focused for developers
- Interested in learning the internals of Kafka
- Interested in building Kafka Clients using Spring Boot
- Interested in building Enterprise standard Kafka client applications using Spring boot

Source Code

Thank You !

Introduction to Apache Kafka

Prerequisites

Course Prerequisites

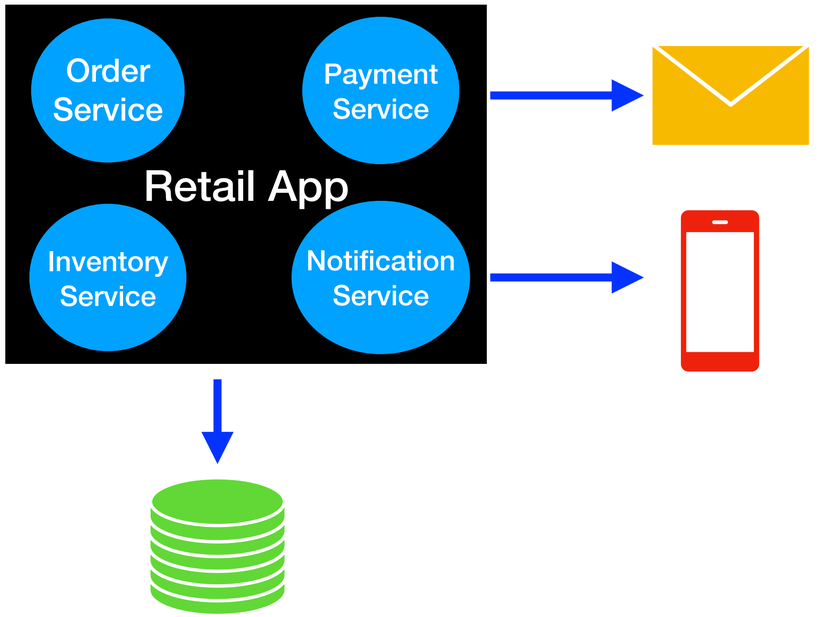
- Prior Knowledge or Working Experience with **Spring Boot/Framework**
- Knowledge about building **Kafka Clients** using Producer and Consumer API
- Knowledge about building **RESTFUL APIs** using Spring Boot
- Experience working with **Spring Data JPA**
- Automated tests using **JUnit5**
- Experience Working with **Mockito**
- **Java 21 or Higher** is needed
- **Intellij** , **Eclipse** or any other IDE is needed

Introduction to Kafka

Understanding the *Why*, *What*, and *Where* of Kafka

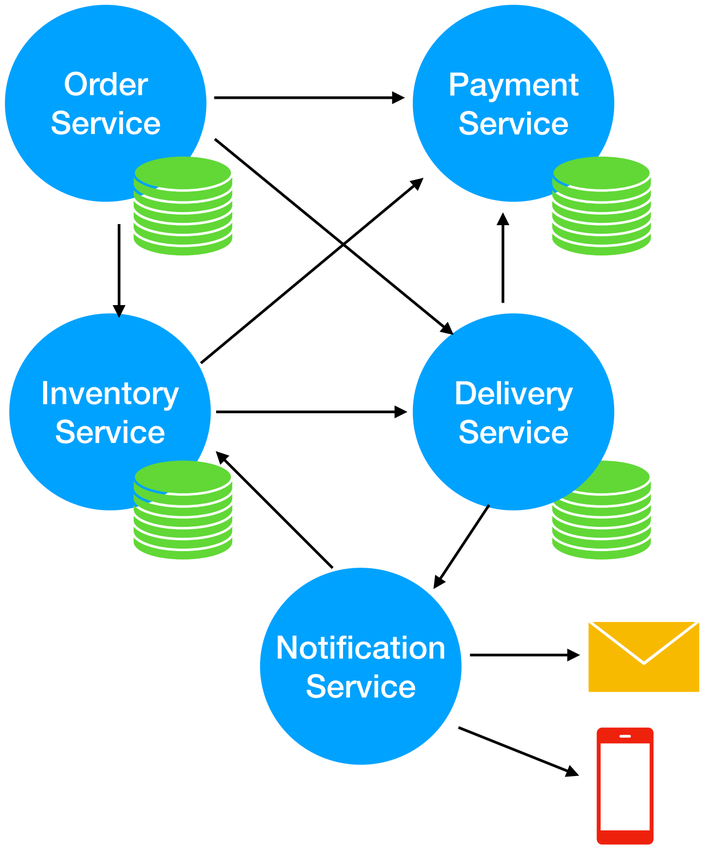
Life Before Kafka

- Monolithic applications
- Synchronous communication
- Tight coupling between systems
- Limited scalability



The Shift to Microservices

- Applications split into smaller services
- Independent deployments
  - Technology diversity
- Need for reliable communication



Early Communication Solutions

- REST APIs
- Traditional message brokers
- Batch-based data processing

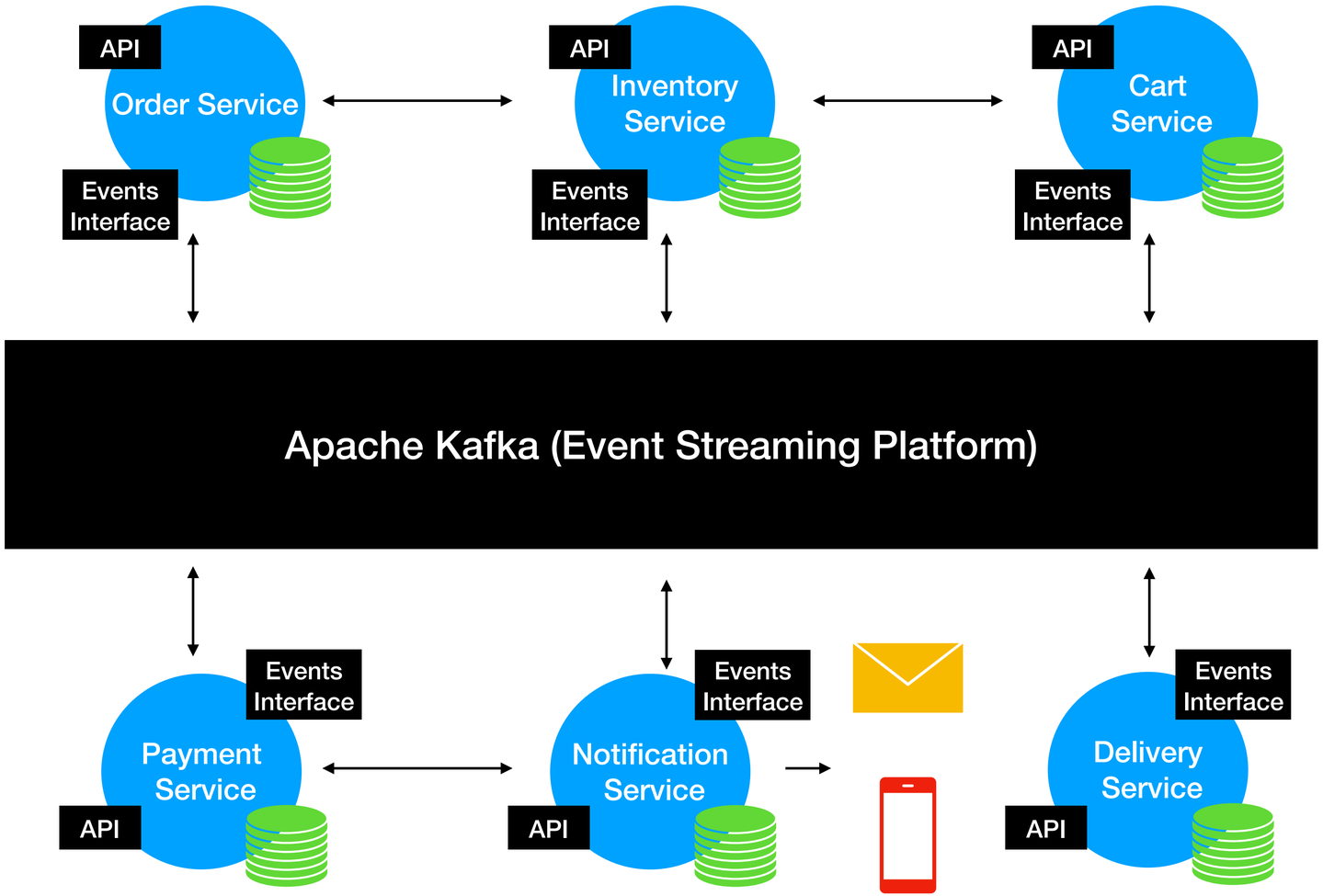
Problems with Traditional Systems

- Tight producer-consumer coupling
- Message loss risk
- Limited scalability
    - Not designed for streaming data

What Is Apache Kafka?

- Distributed event streaming platform
- Kafka Features:
  - High-throughput
  - Fault-tolerant
  - Horizontally scalable
- Kafka is not just a messaging system — it’s a **commit log for events**

MicroServices Architecture

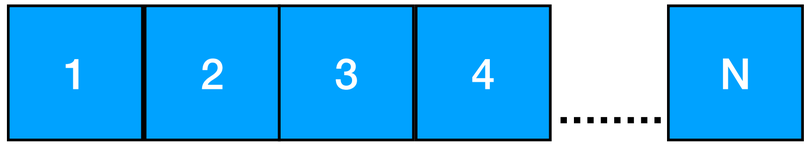


Kafka’s Core Idea

- Producers and Consumers subscribe to a stream of records



- Store stream of Events



Kafka vs Traditional Messaging

- Persistent storage vs transient messages
- Pull-based consumption
- Replay capability
- Designed for scale

Kafka Use Cases

Transportation



Driver-Rider Notifications

Food Delivery Notifications

Retail



Sale Notifications

RealTime Purchase recommendations

Tracking Online Order Deliveries

Banking



Fraud Transactions

New Feature/Product notifications

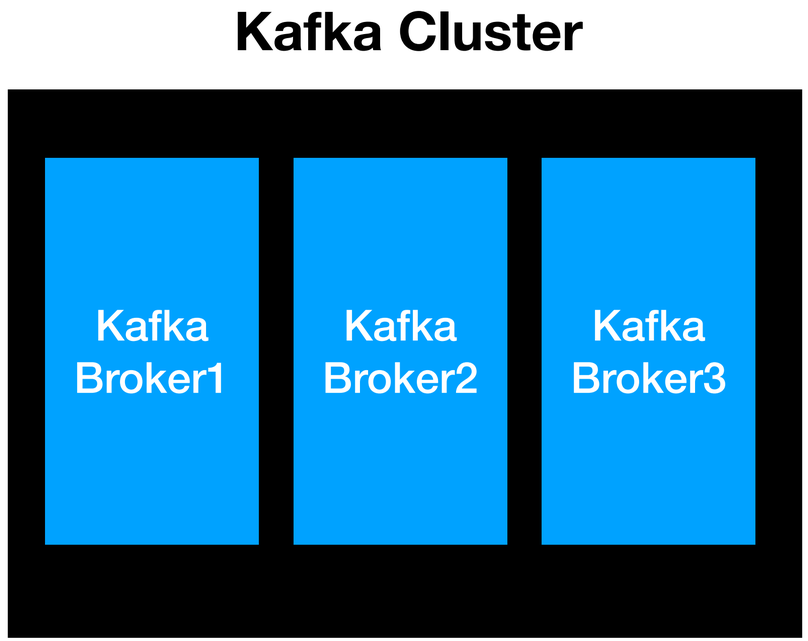
Why Kafka Matters

- Loose coupling
- Scalability
- Reliability
- Real-time processing

Kafka Core Terminologies
[A Quick Overview]

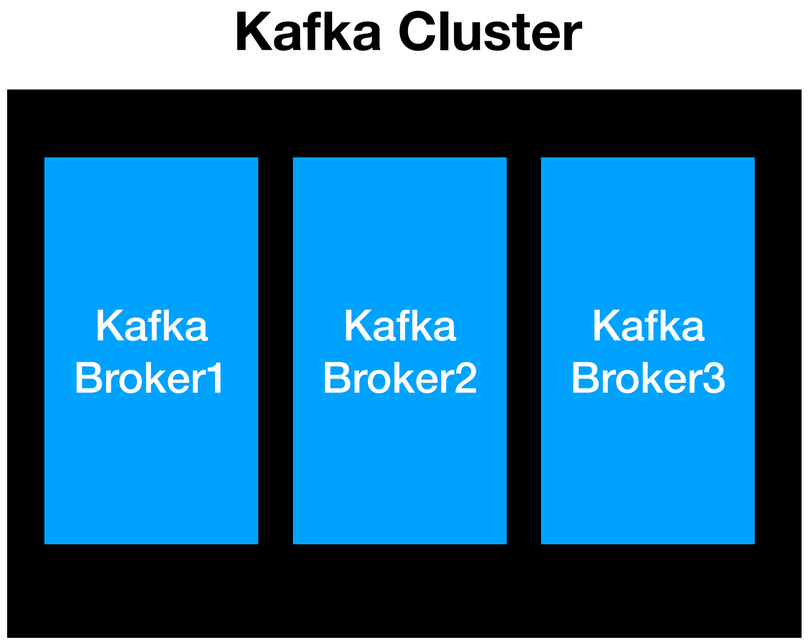
Kafka Cluster

- Kafka runs as a cluster
- Multiple servers working together
- Designed for fault tolerance



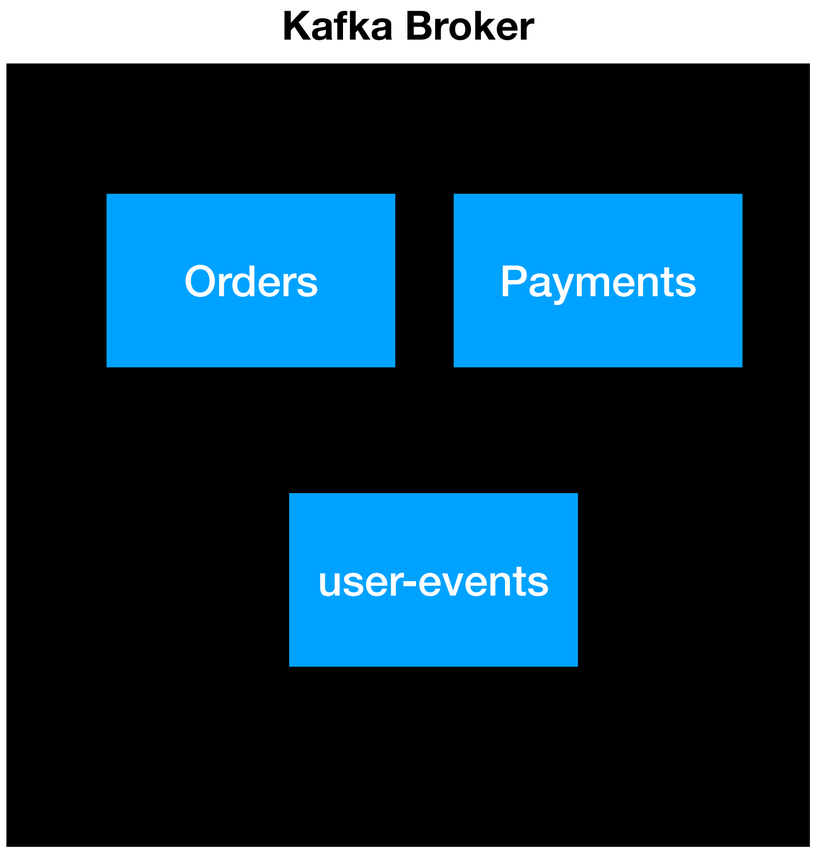
Kafka Broker

- Broker = Kafka server
- Stores data on disk
- Serves producers & consumers



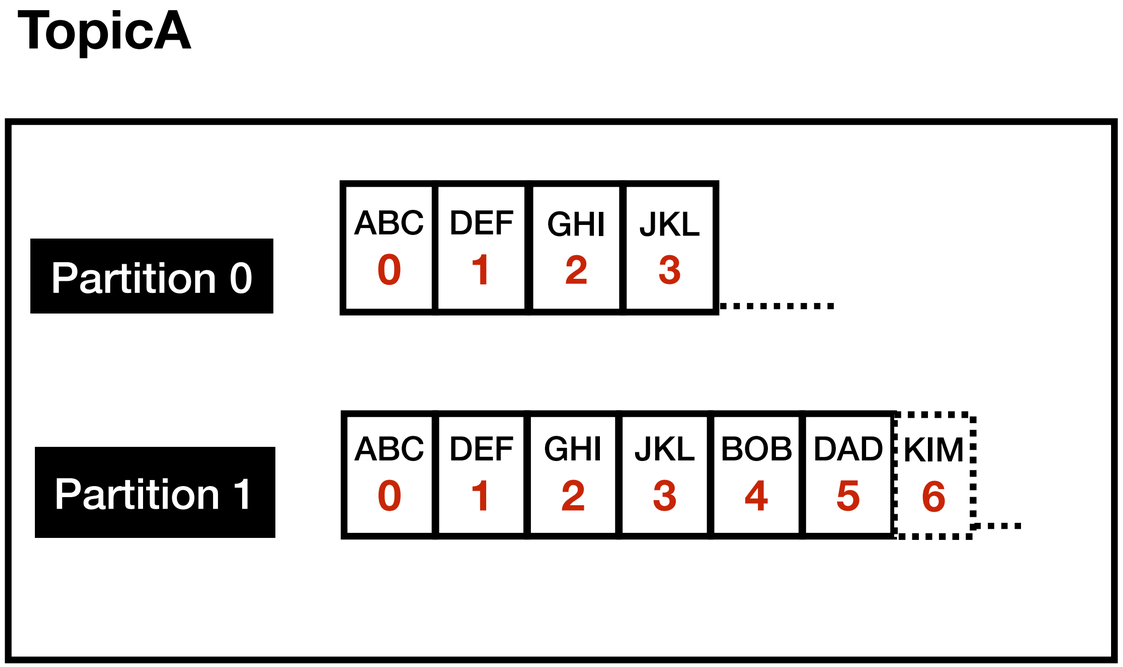
Topic

- Logical stream of events
- Named data category
- Append-only log



Partition

- Topic is split into partitions
- Enables parallelism
- Ordering per partition



Producer

- Publishes records to topics
- Chooses partition (optional)
- Writes are append-only

Consumer

- Reads records from topics
- Pull-based model
- Controls read speed

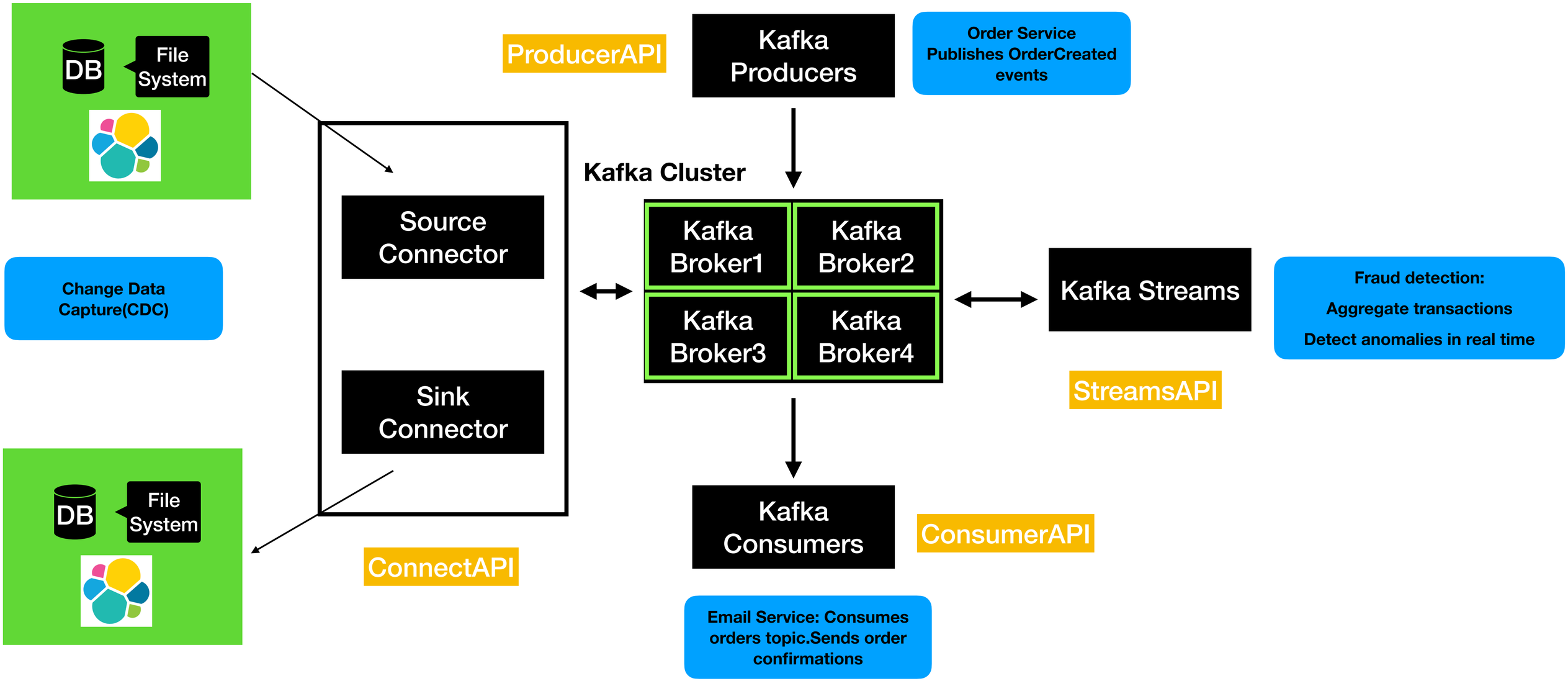


Kafka APIs

Different ways to interact with Kafka

- Producer API → Write events to Kafka
- Consumer API → Read events from Kafka
- Admin API → Manage Kafka infrastructure
- Kafka Streams API → Process Kafka data in real time
- Kafka Connect API → Move data between Kafka and other systems

Kafka Client APIs + UseCases



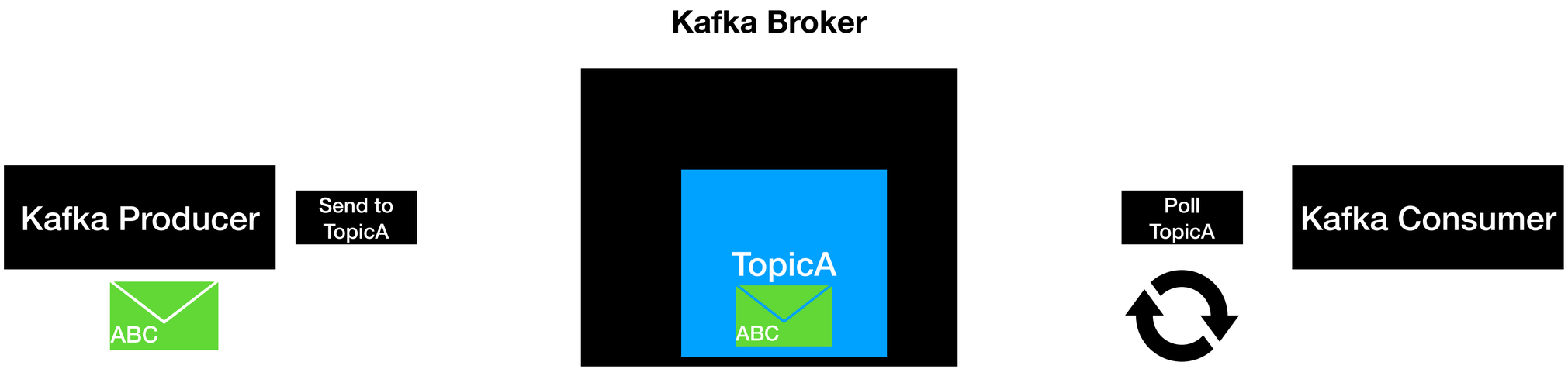
Kafka Topics
&
Partitions

Kafka Topics

- Topic is an **Entity** in Kafka with a name

Kafka Topics

- Topic is an **Entity** in Kafka with a name

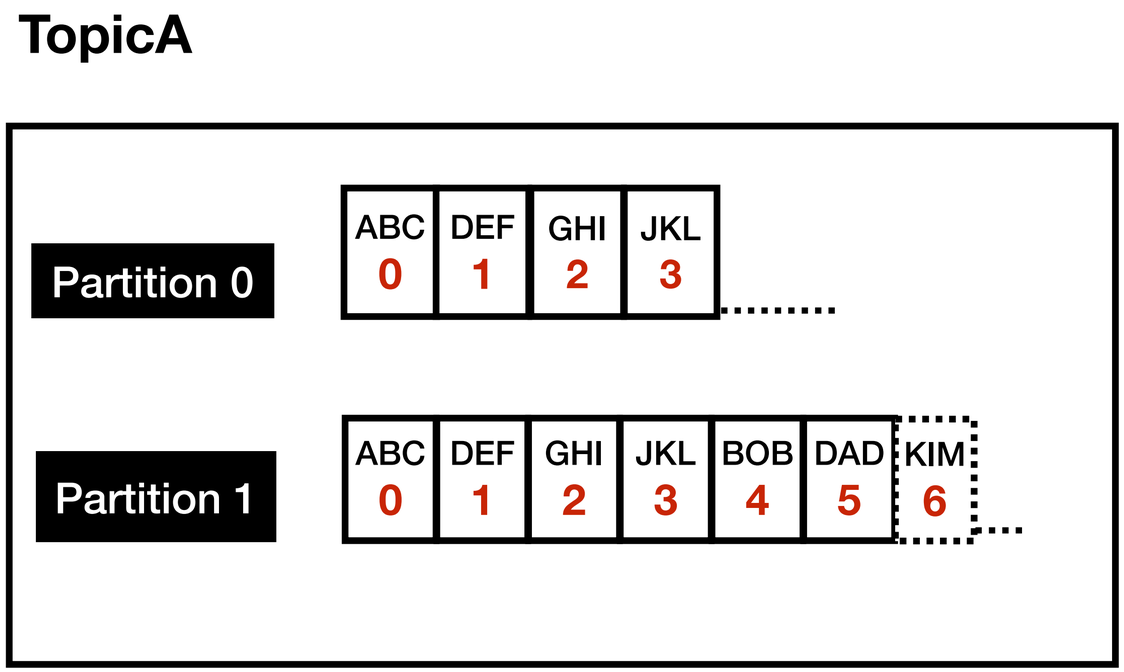


Topic and Partitions

- Partition is where the message lives inside the topic

- Each Topic will be create with one or more partitions

Topic and Partitions



- Each Partition is an ordered , immutable sequence of records

- Each record is assigned a sequential number called ***offset***

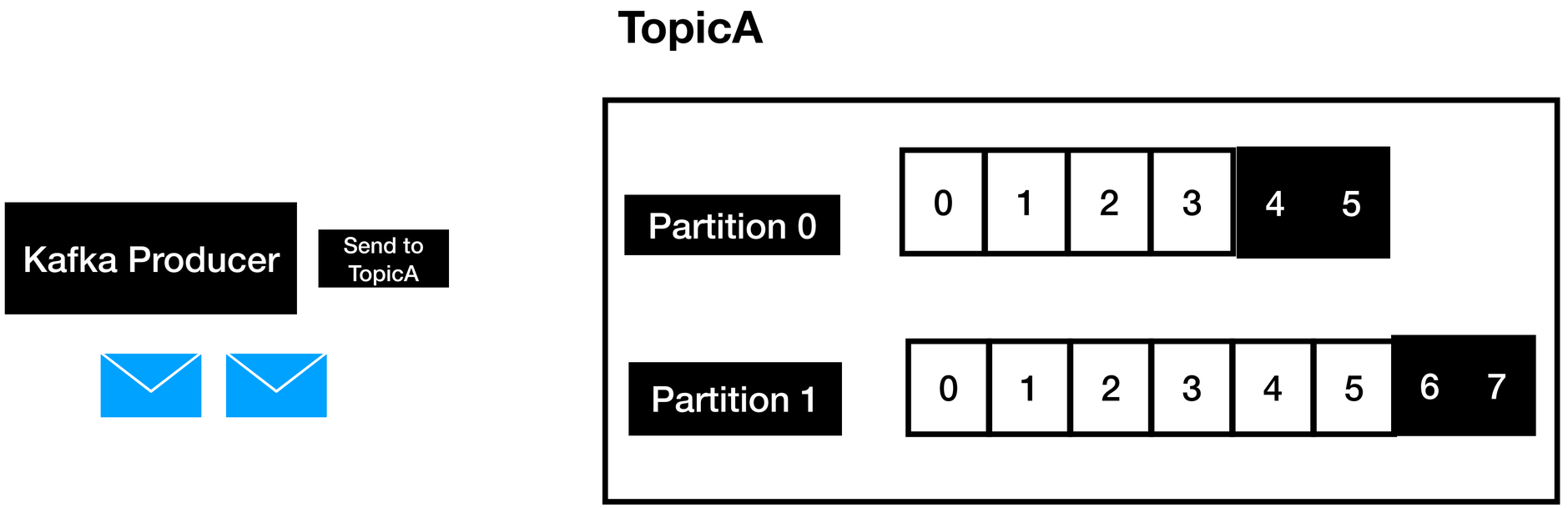
- Each partition is independent of each other

- Ordering is guaranteed only at the partition level

- Partition continuously grows as new records are produced

- All the records are persisted in a commit log in the file system where Kafka is installed

Topics and Partitions



Setting up Kafka Broker [using **KRaft**]

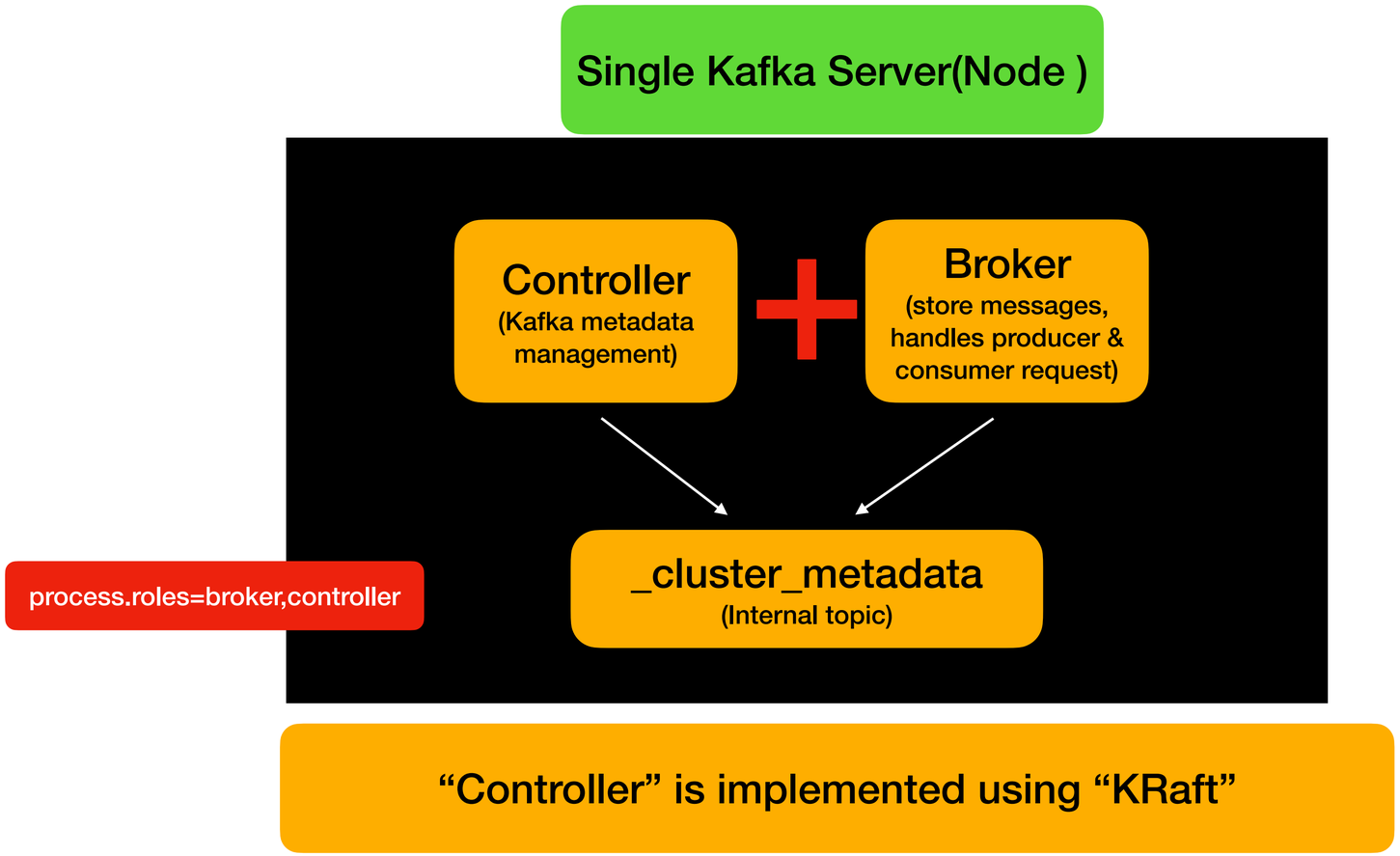
Kafka Needs a Brain : Controller

- A Kafka broker by itself only knows how to:
  - Store messages on disk
  - Send messages to consumers
  - Receive messages from producers
- But a broker doesn't inherently know:
  - What topics exist in the system
  - How many partitions each topic has
  - Which brokers are alive or dead

This is where “Controller” comes into play.

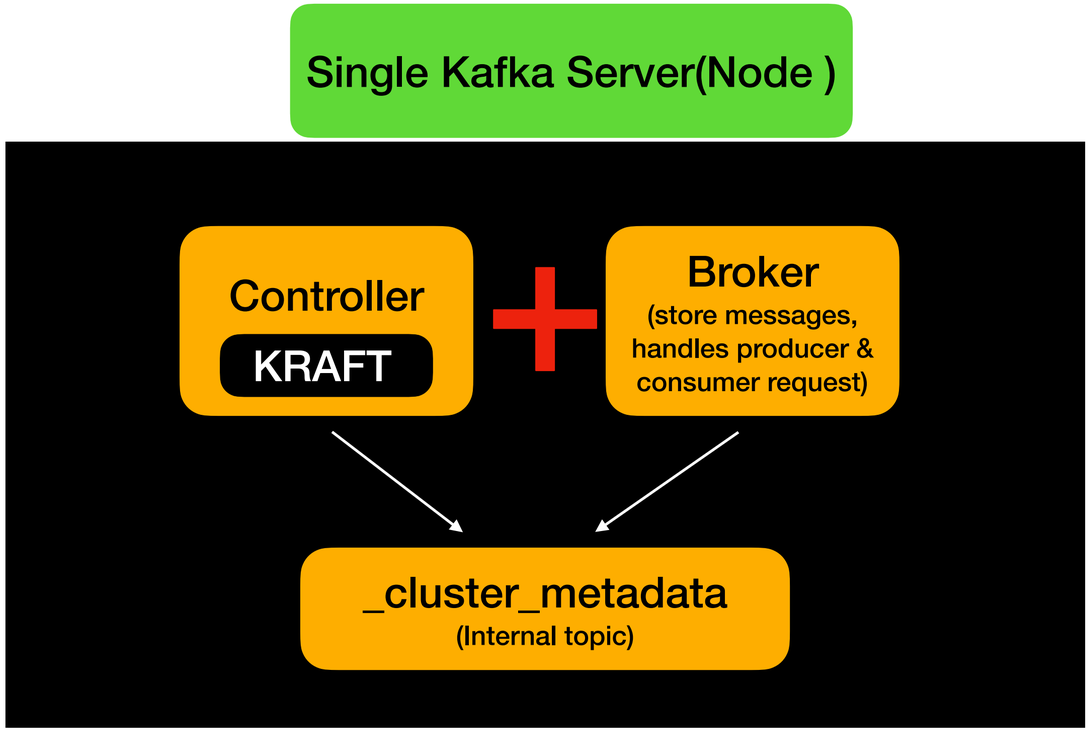
How is Controller setup in Kafka?

- Controller is just a Kafka server with additional responsibilities.



What is KRaft (aka as Controller) ?

- KRaft (Kafka Raft) is Kafka's built-in consensus protocol that replaces Apache Zookeeper for metadata management.
- KRAFT = Kafka + Raft



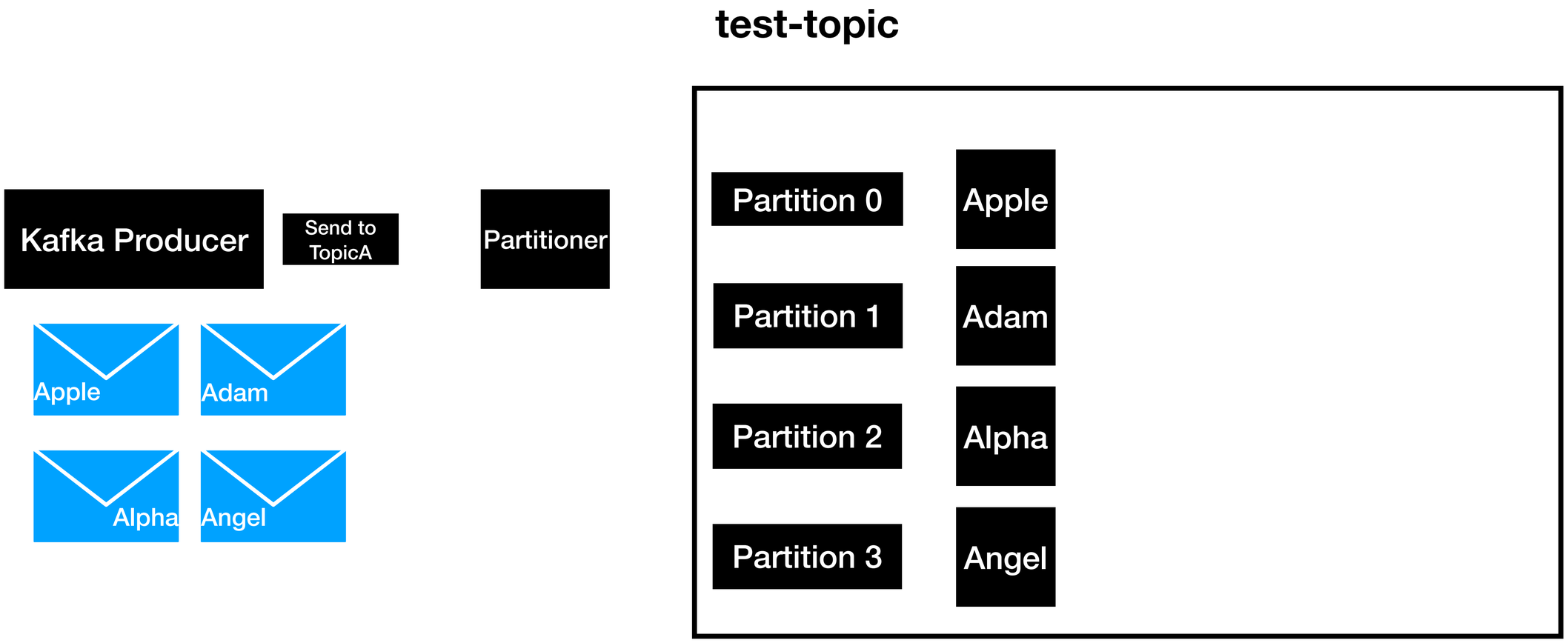
Kafka Server Startup - Behind the scenes

Sending Kafka Messages With Key and Value

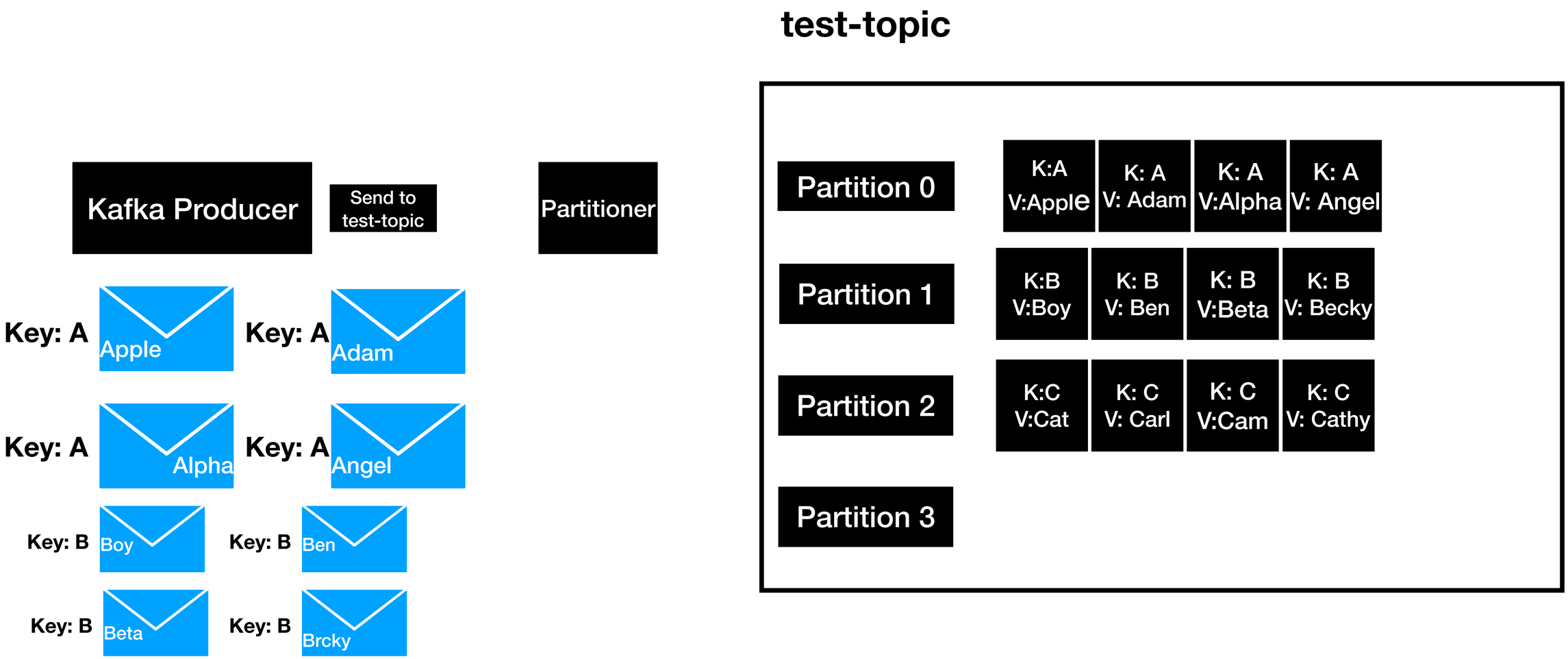
Kafka Message

- Kafka Message these sent from producer has two properties
  - Key (optional)
  - Value

Sending Message Without Key



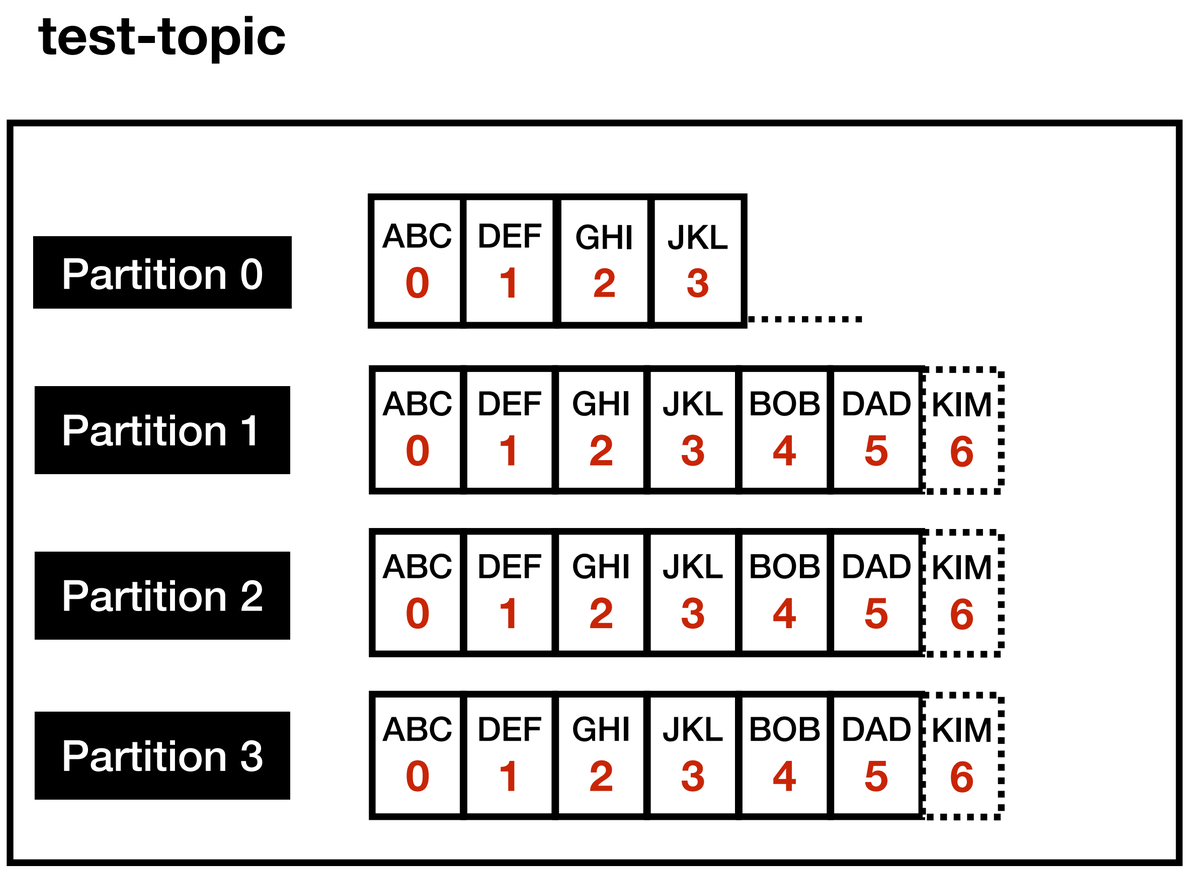
Sending Message With Key



Consumer Offsets

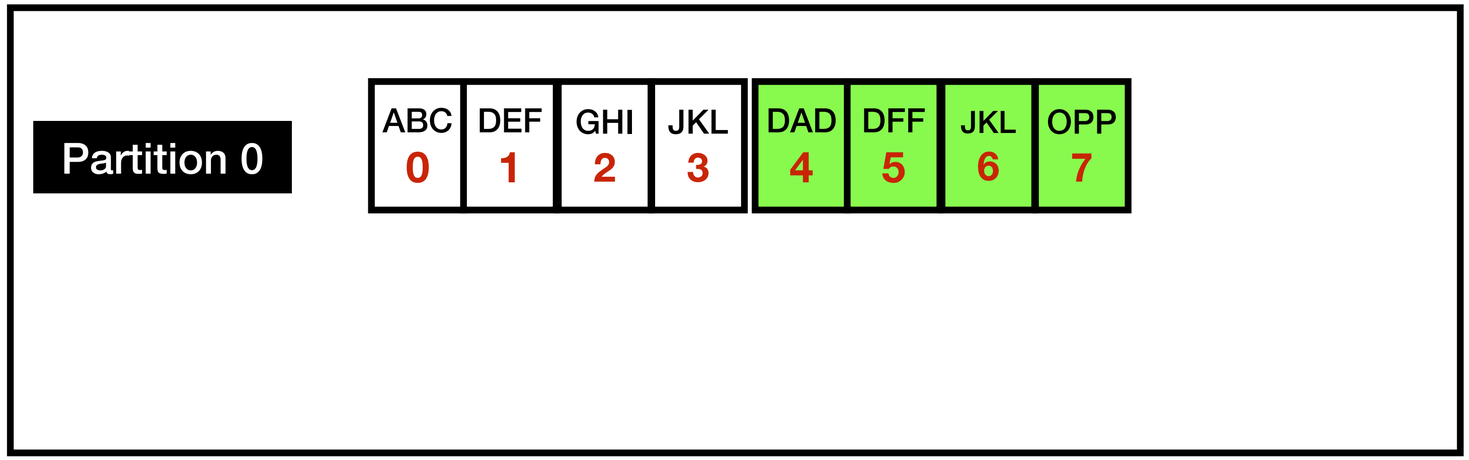
Consumer Offsets

- Consumer have three options to read
  - from-beginning
  - latest
  - specific offset



Consumer Offsets

test-topic



From beginning

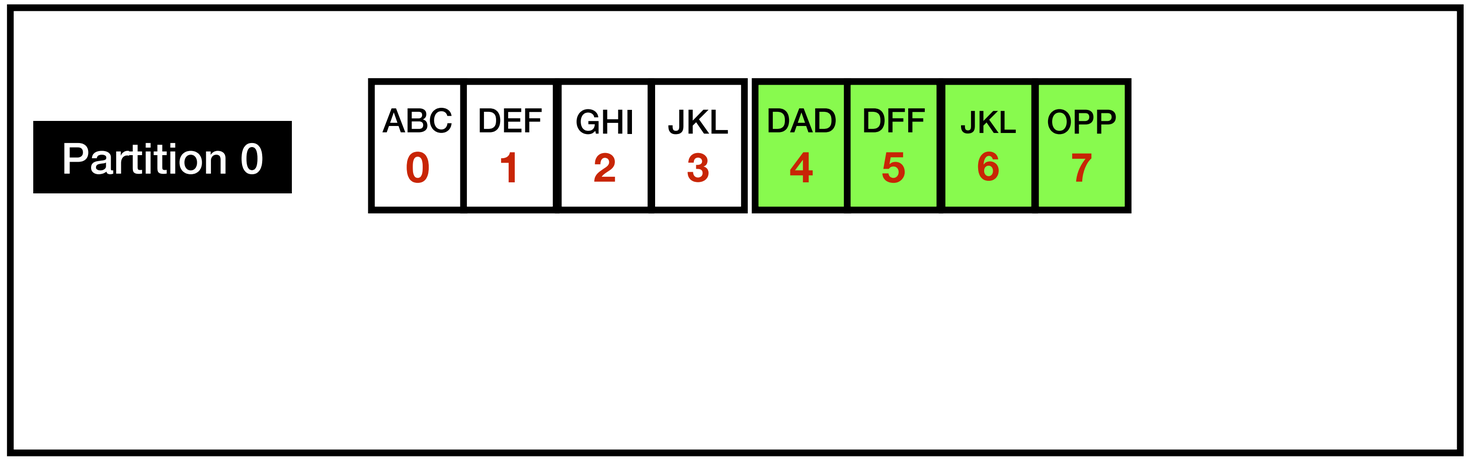
group.id = group1

Consumer 1

__consumer_offsets

Consumer Offsets

test-topic



From beginning

group.id = group1

Consumer 1

__consumer_offsets

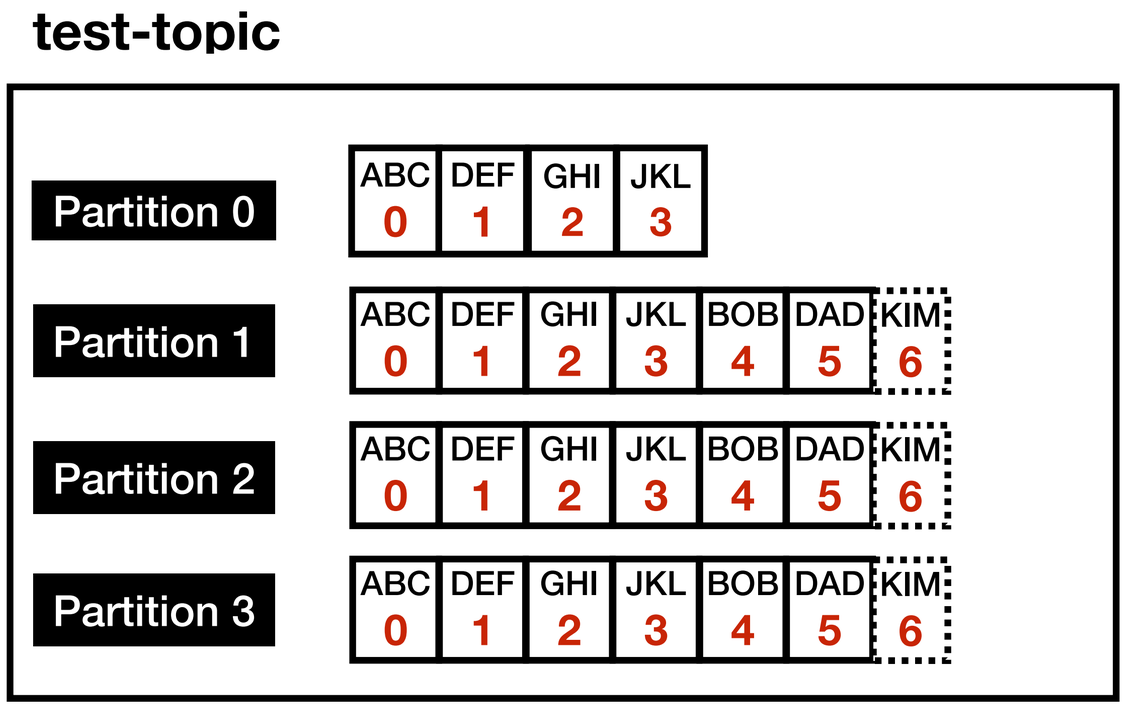
Consumer Offset

- Consumer offsets behaves like a bookmark for the consumer to start reading the messages from the point it left off.

Consumer Groups

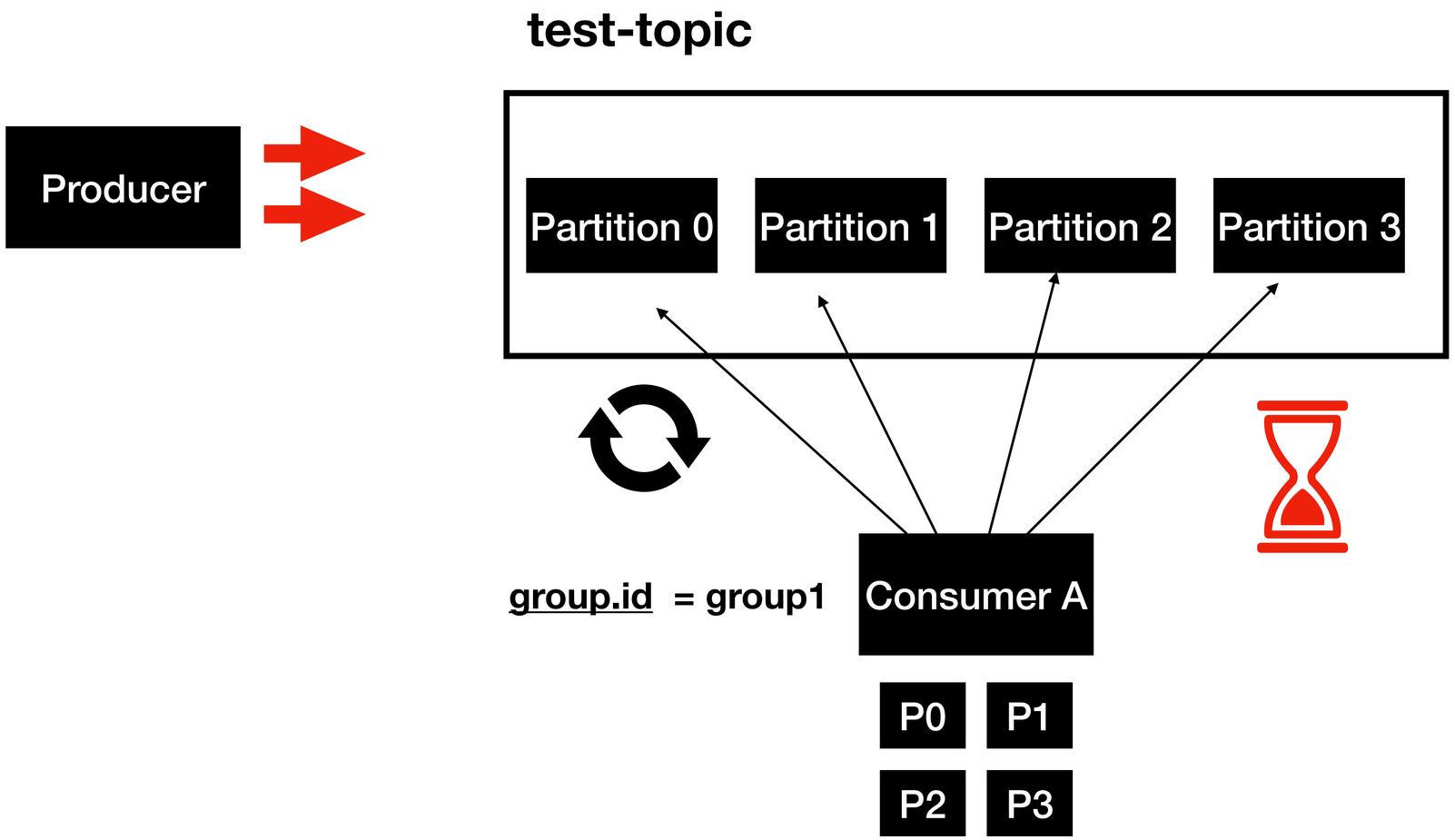
Consumer Groups

- **group.id** is mandatory
- **group.id** plays a major role when it comes to scalable message consumption.

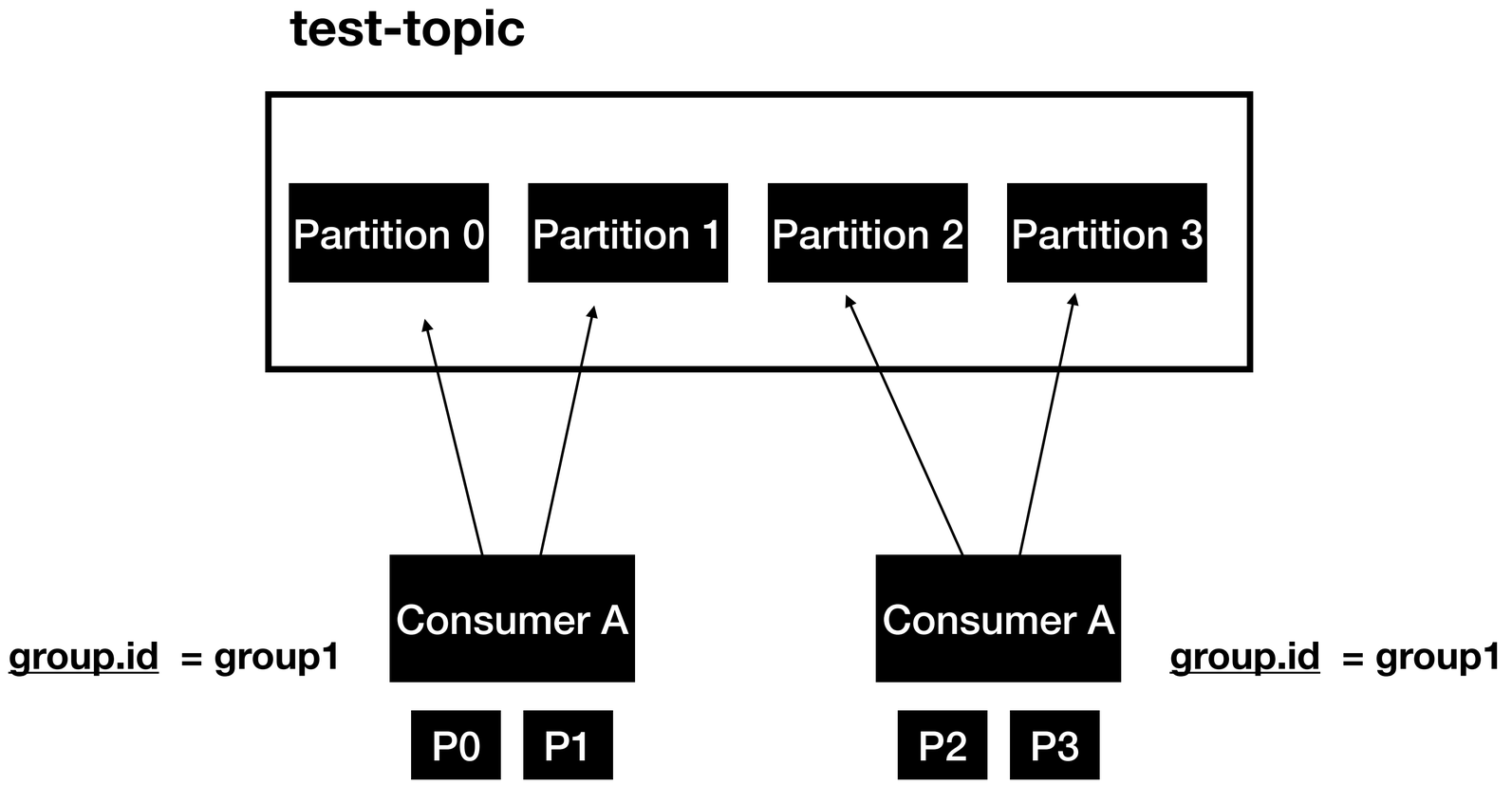


group.id = group1 Consumer 1

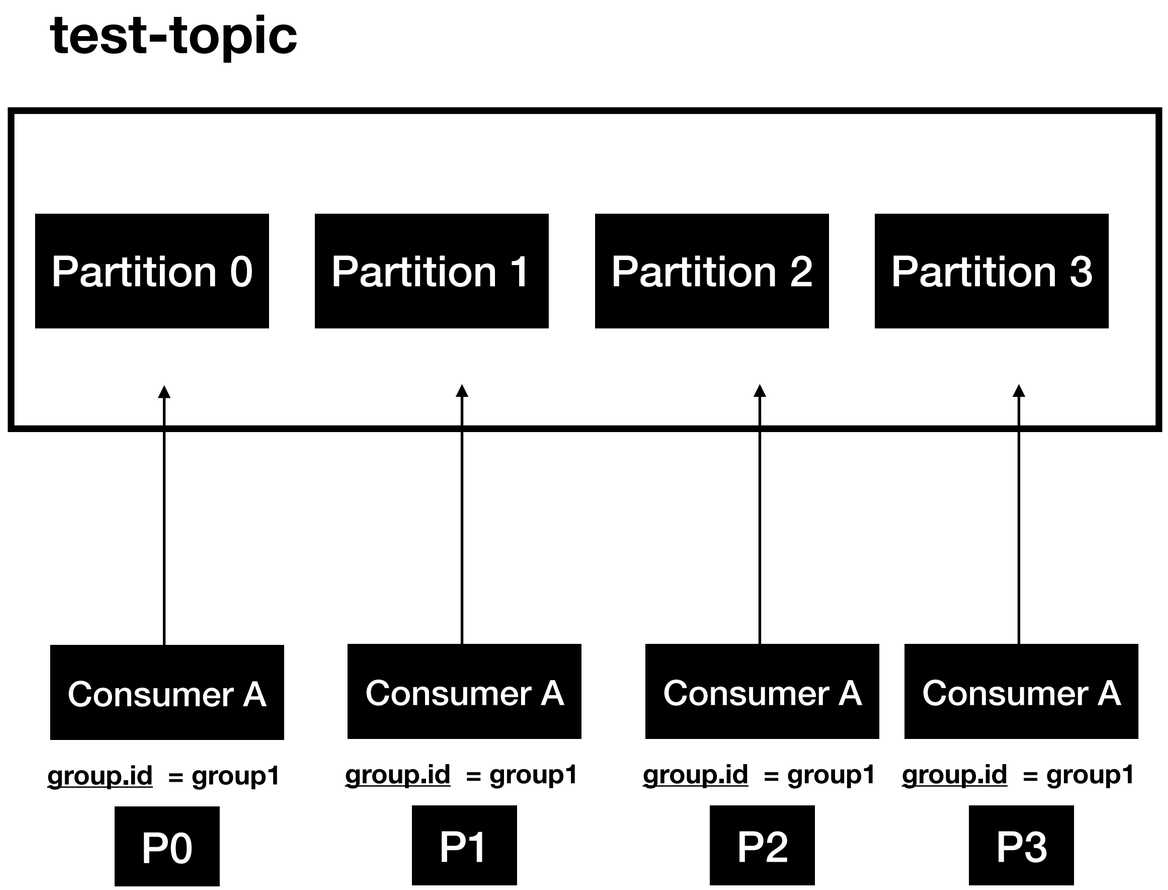
Consumer Groups



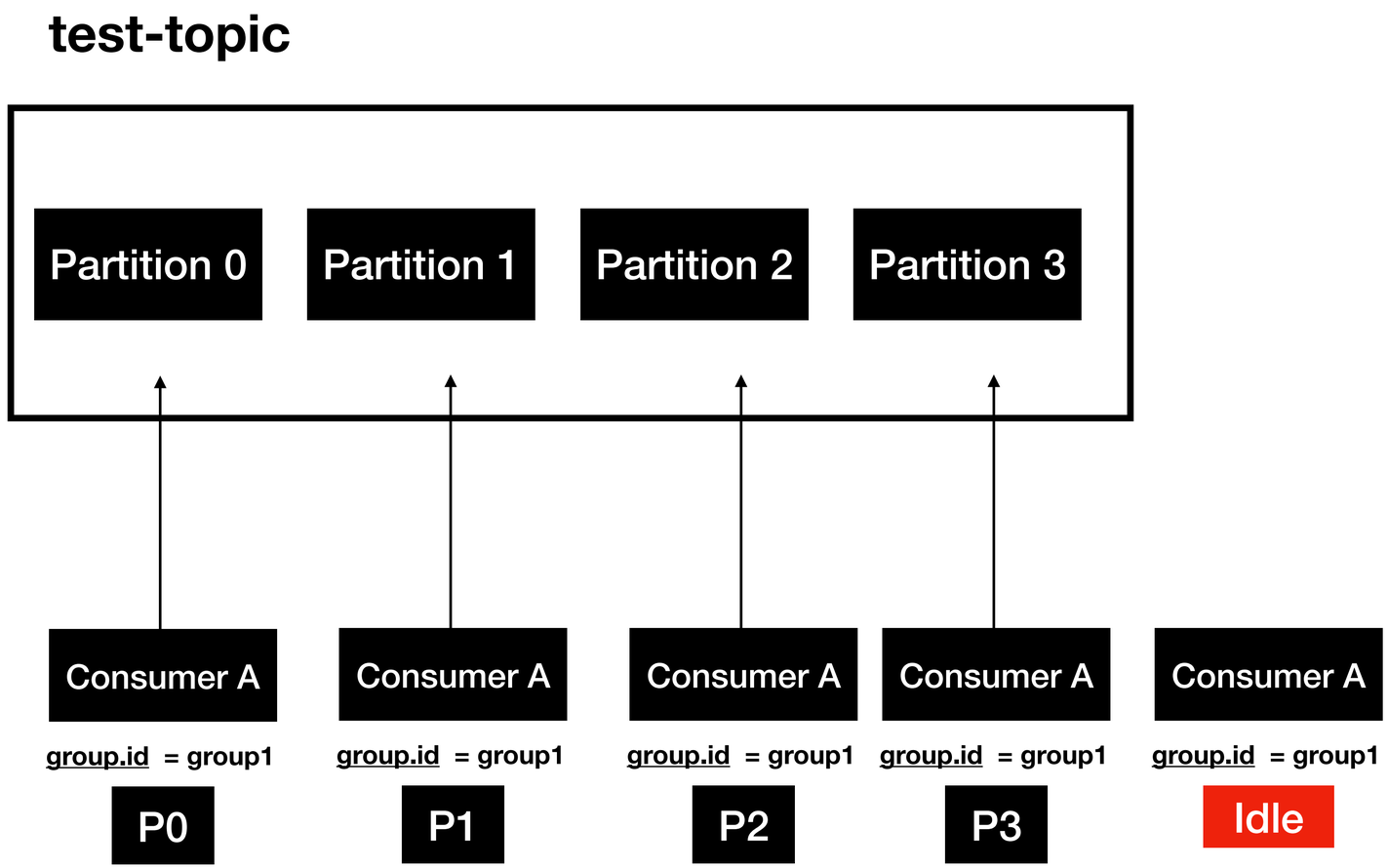
Consumer Groups



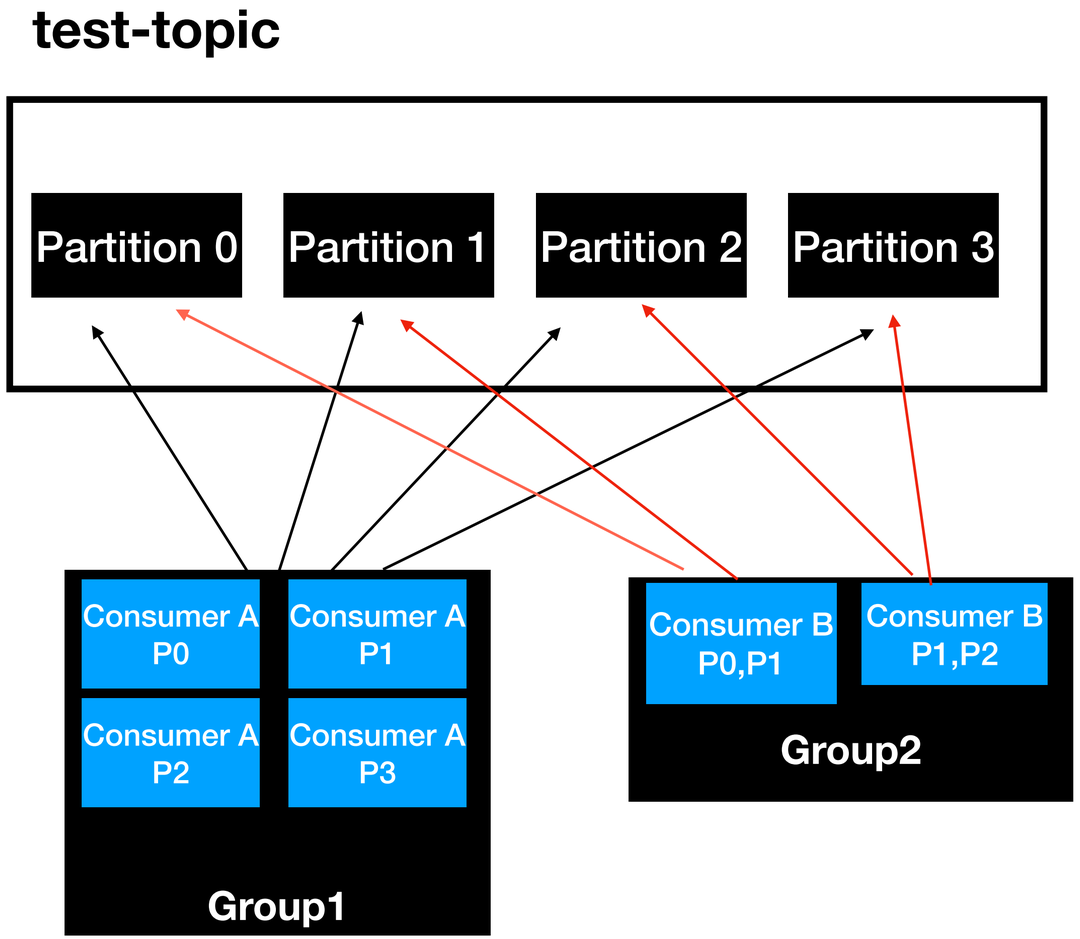
Consumer Groups



Consumer Groups



Consumer Groups

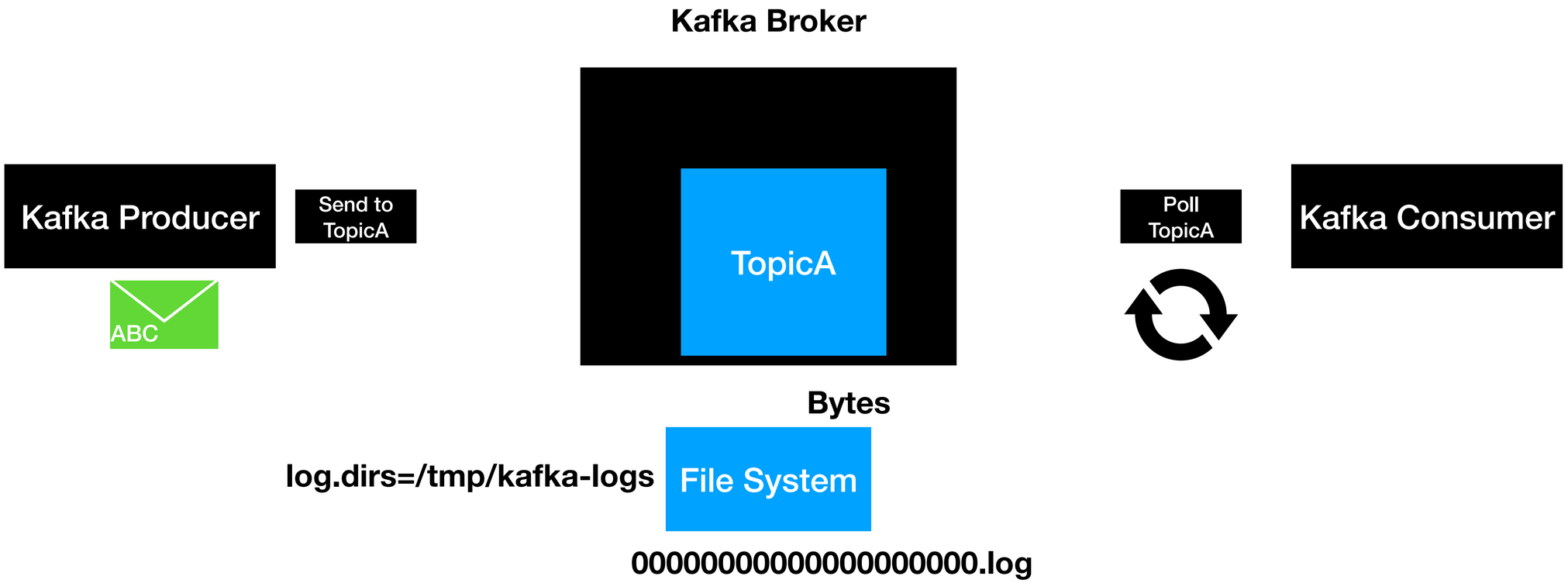


Consumer Groups : Summary

- Consumer Groups are used for scalable message consumption
- Each different application will have a unique consumer group
- Who manages the consumer group?
  - Kafka Broker manages the consumer-groups
  - Kafka Broker acts as a Group Co-ordinator

Commit Log
&
Retention Policy

Commit Log


Kafka Broker
Kafka Producer
Send to TopicA
ABC
TopicA
Poll TopicA
Kafka Consumer
Bytes
log.dirs=/tmp/kafka-logs
File System
00000000000000000000.log

Retention Policy

- Determines how long the message is retained ?
- Configured using the property **log.retention.hours** in **server.properties** file
- Default retention period is **168 hours** (7 days)

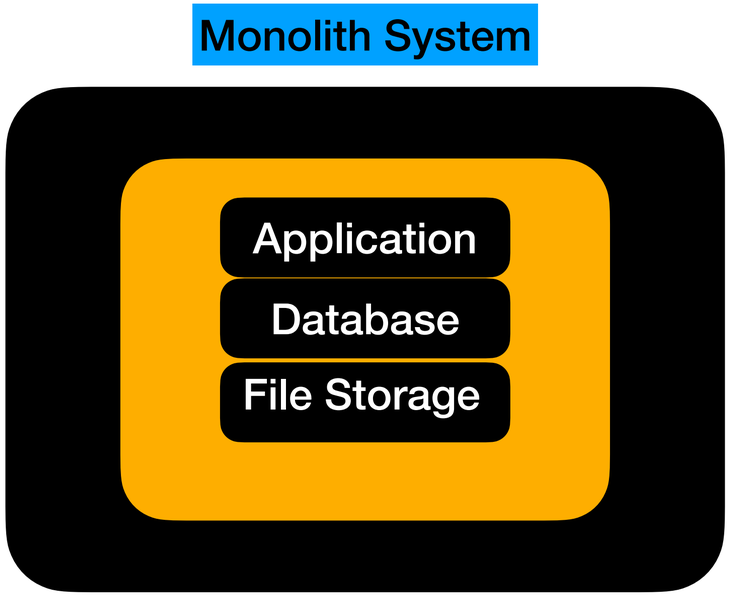
Kafka
as a
Distributed Streaming System

Apache Kafka® is *a distributed streaming platform*

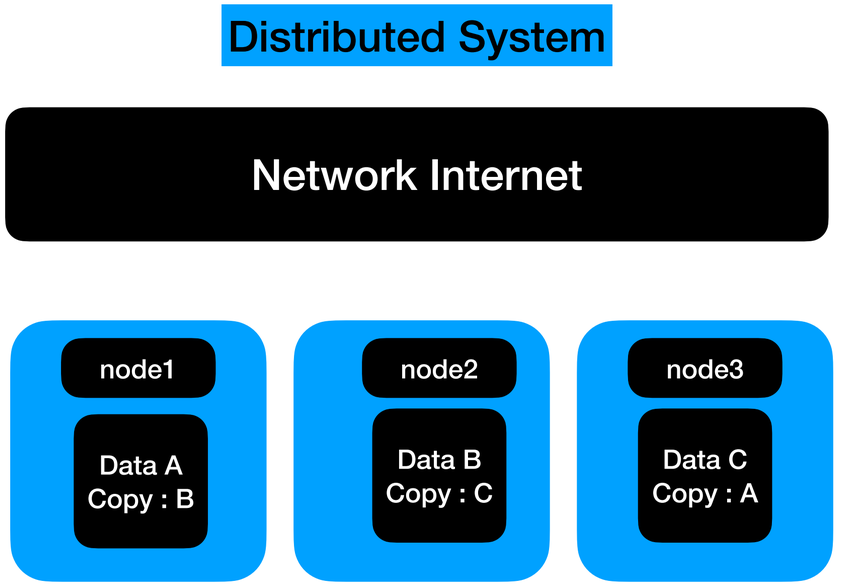
What is a Distributed System ?

- A **distributed system** is a collection of independent computers (nodes) that communicate over a network and coordinate their actions to appear as a single coherent system to end users.
- Real-World Examples of Distributed Systems
  - Google Search - Query hits one of thousands of servers; index split across data centers worldwide.
  - Netflix - Content cached on edge servers globally; recommendation engine runs on hundreds of nodes.
- Think of it like a restaurant kitchen:
  - A single chef (monolith) can only cook so many dishes.
- Multiple chefs (distributed) working together can serve hundreds of customers.
  - They coordinate, share ingredients, and cover for each other if someone is sick.

Monolith vs Distributed Systems



- Single Point of Failure
- Limited Scalability
- Maintenance Downtime



Appears as one system to the user

- No Single Point of Failure
- Horizontal Scalability - add more nodes as needed.
- Parallel Processing - work divided across nodes.

Why Do We Need Distributed Systems?

- **SCALE**
  - Modern applications handle:
  - Facebook: 500,000+ requests/second
  - Google: 99,000+ searches/second
  - Kafka at LinkedIn: 7 trillion messages/day
- **RELIABILITY**
  - Hardware fails. It's not "if" but "when":
  - Hard drives: 2-4% annual failure rate
  - Servers: ~5% annual failure rate in large data centers
  - At 10,000 servers: ~1-2 failures per day!
- **LATENCY**
  - Speed of light limits us:
  - Users expect <100ms response times
  - Solution: Put servers close to users (CDNs, edge computing)
- **DATA LOCALITY**
  - Laws like GDPR require data to stay in certain regions:
  - European user data must stay in EU
  - Distributed systems can enforce this

Scalability in Distrbuted Systems

VERTICAL SCALING
(Scale Up)

Before



4 CPU
8 GB RAM
500 GB Disk

After

32 CPU
256 GB RAM
4 TB Disk

- Upgrade Single Server
- More CPU, RAM, Storage
- Hardware Limits

HORIZONTAL SCALING
(Scale Out)

Before

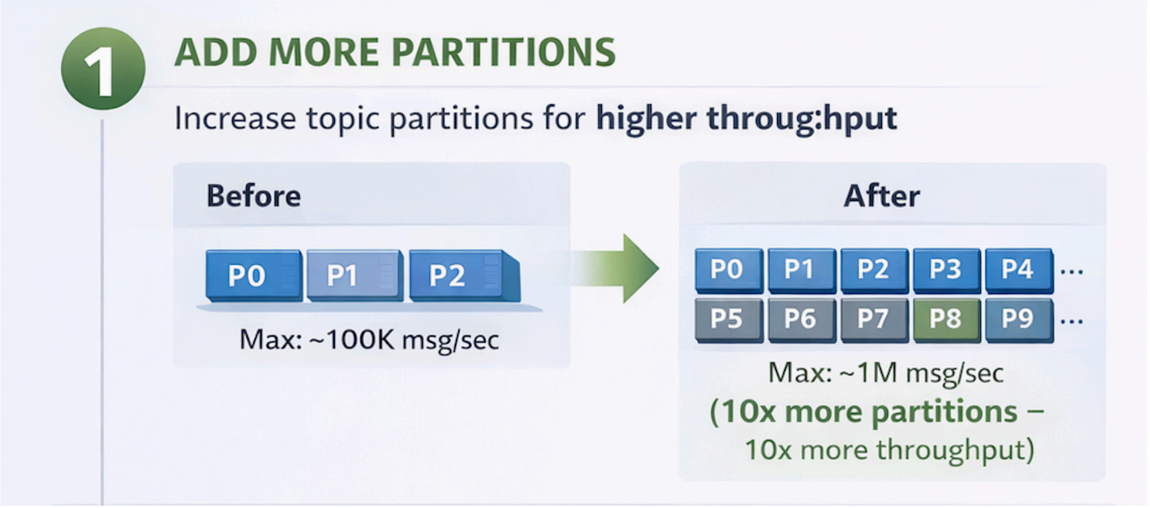
After

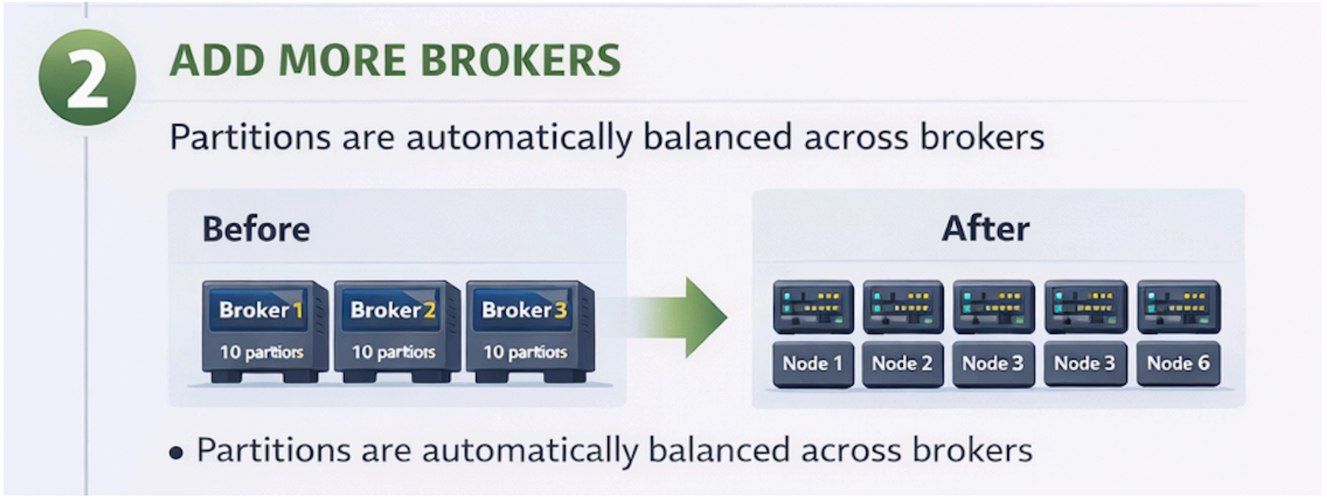


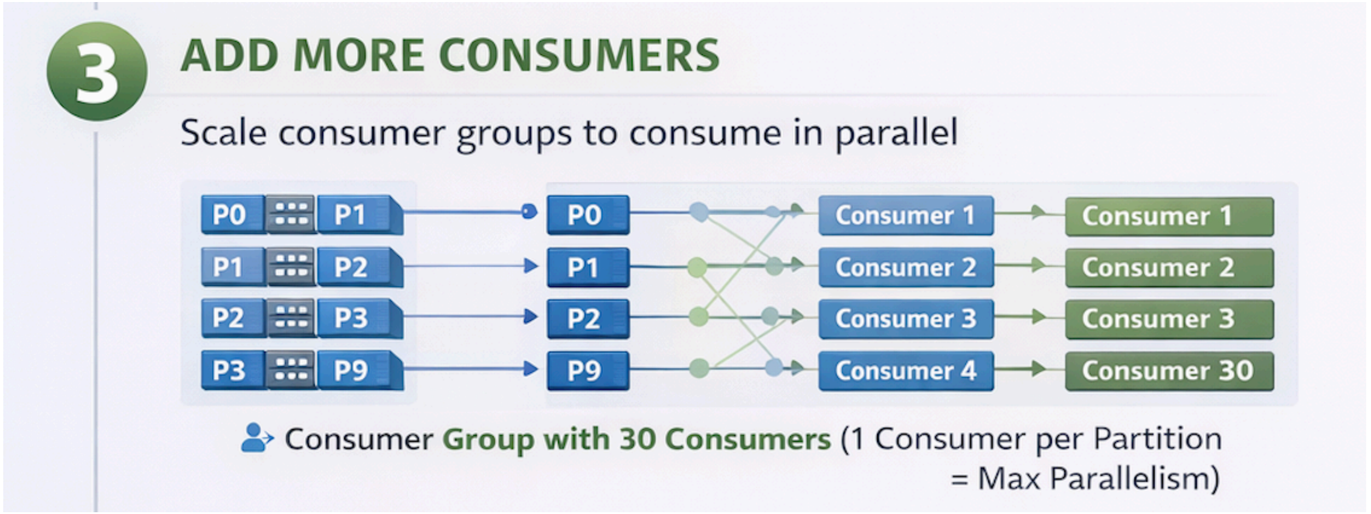
- Add More Servers
- Distributed System
- High Availability

Scalability in Kafka

Imagine your application grows from **1,000 messages per second to 1,000,000 messages per second**.

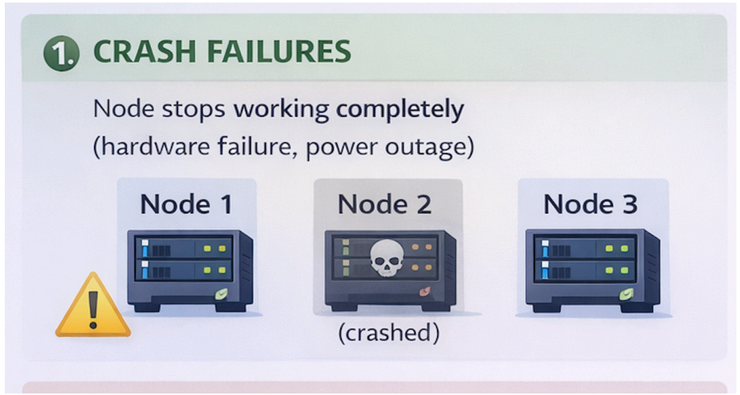


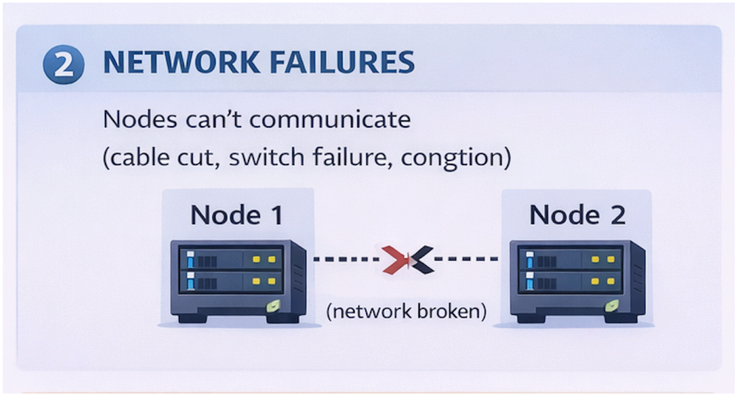


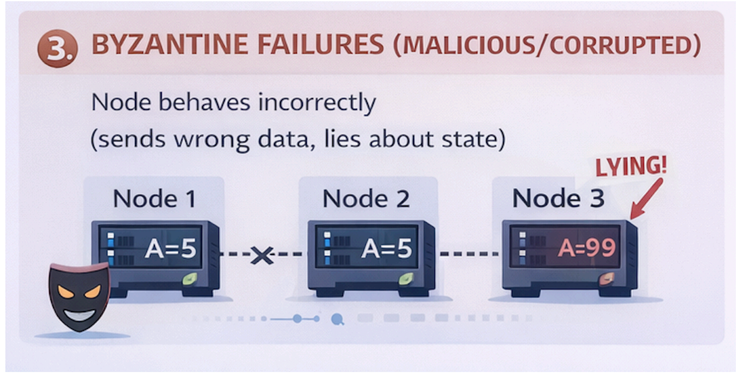


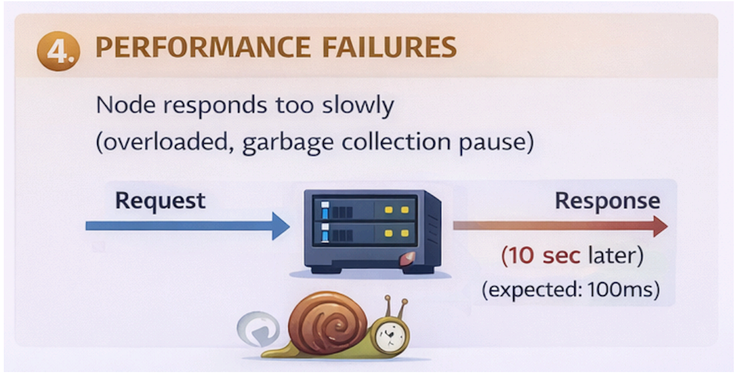
Fault Tolerance in Distributed Systems

- Fault tolerance is the ability of a system to continue operating correctly even when some of its components fail.
  - And this is extremely important because in distributed systems, failures are normal, not exceptional.

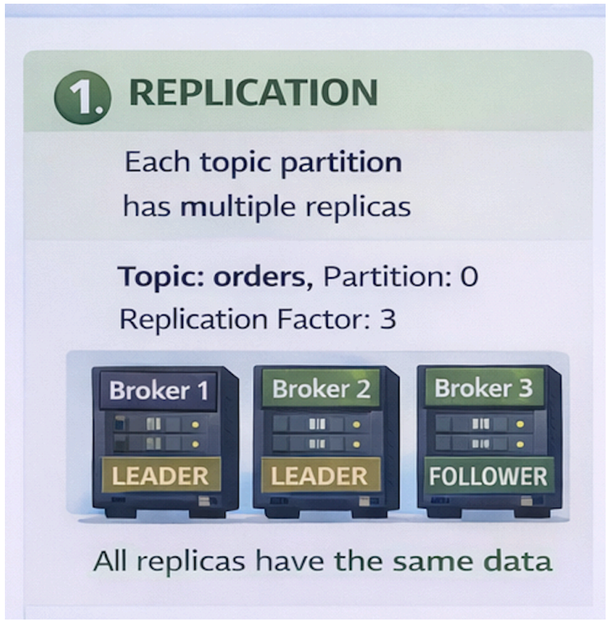


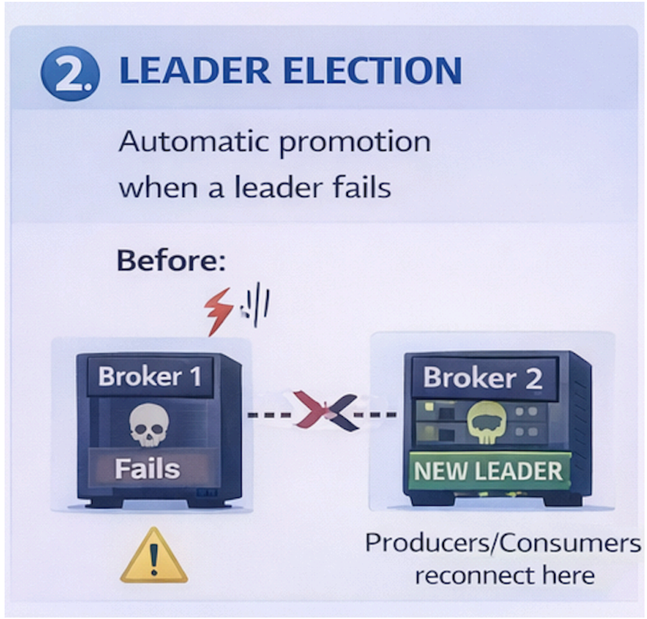


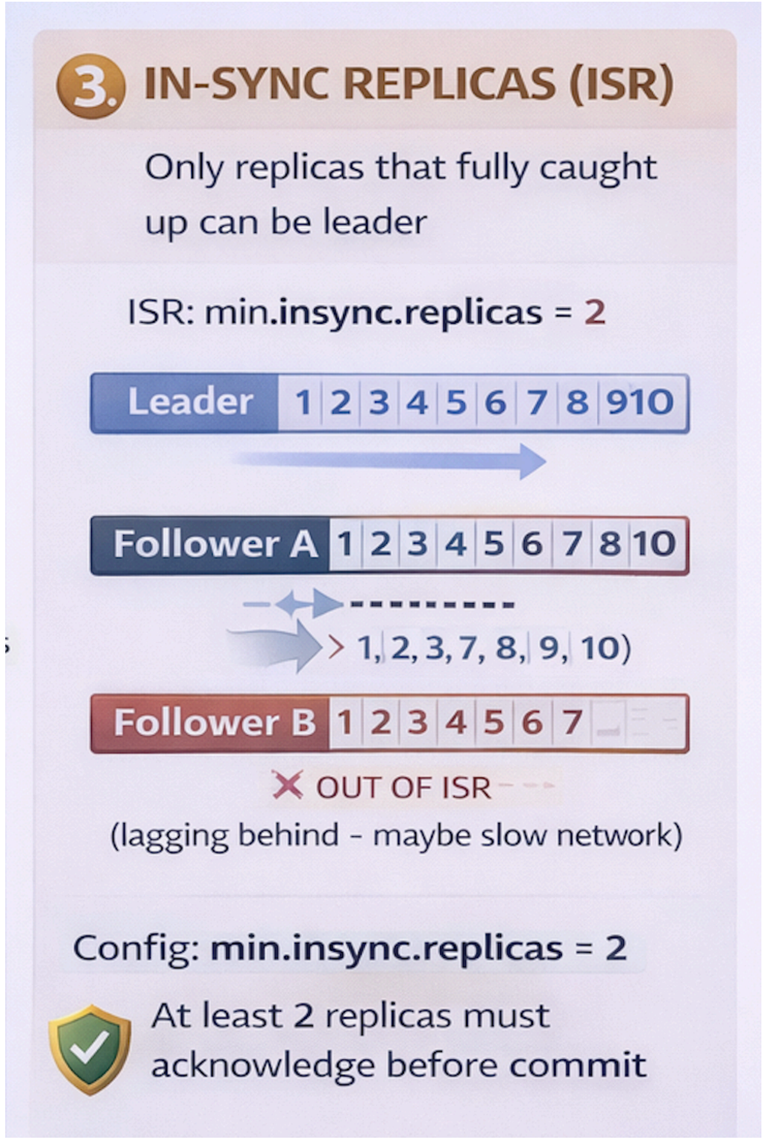




How Kafka Handles Failures?







High Availability in Kafka

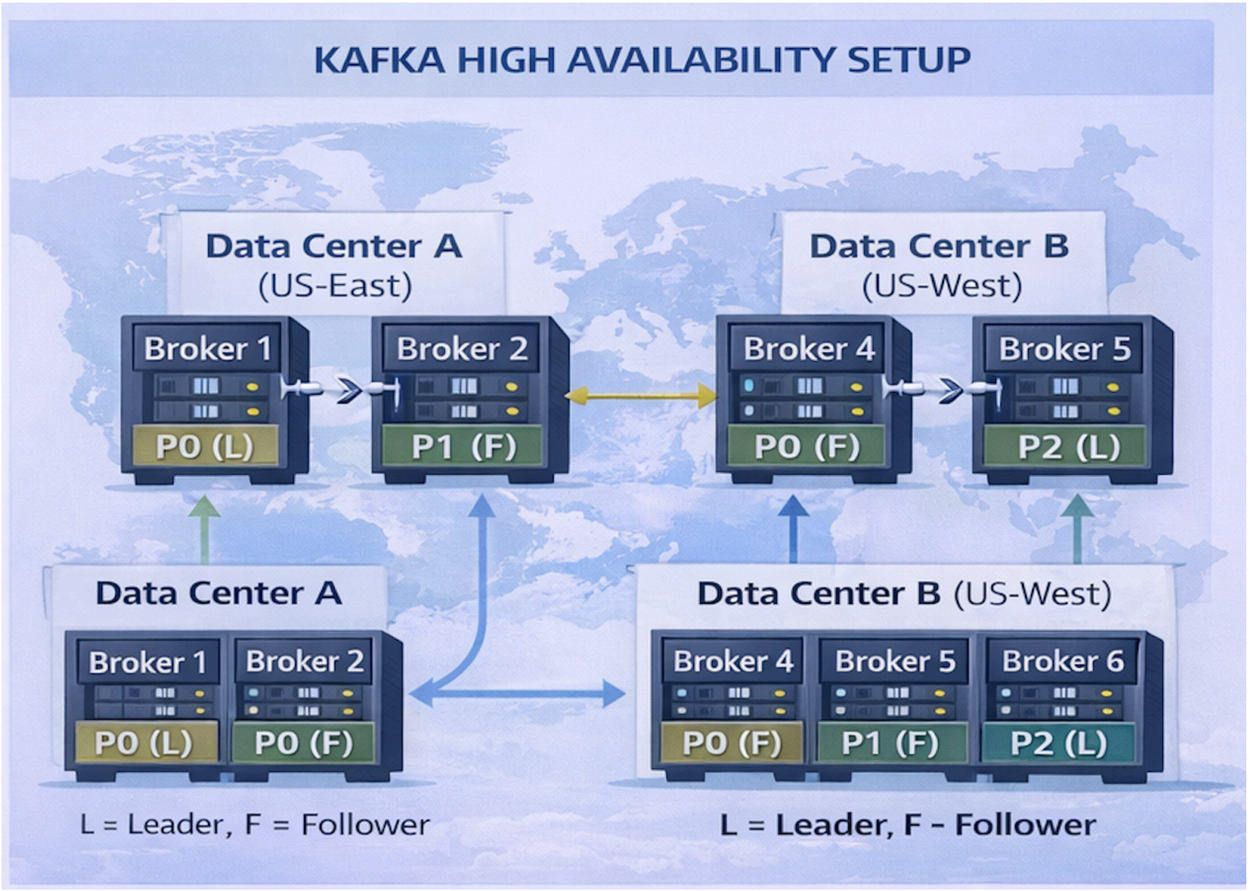
- What is High Availability?
  - High availability means that a system remains operational and accessible for a very high percentage of time.
-

1. AVAILABILITY LEVELS (THE "NINES")

Availability	Downtime/Year	Downtime/Week
90%	36.5 days	3 days
99%	3.65 days	7.3 hours
99.9%	8.76 hours	43.8 minutes
99.99%	52.6 minutes	4.4 minutes
99.999%	5.26 minutes	26.3 seconds
99.9999%	31.5 seconds	2.6 seconds

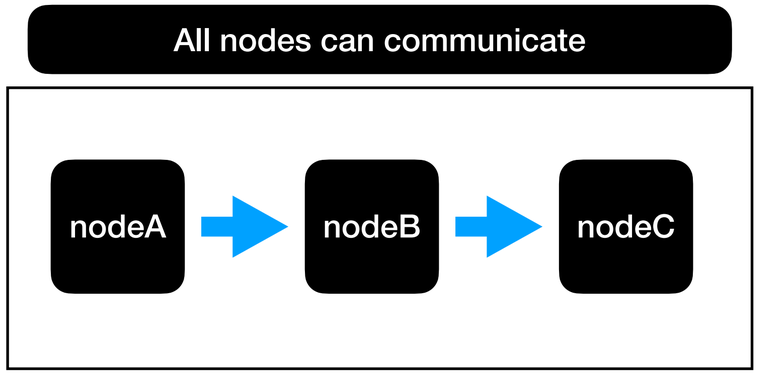
REAL-WORLD EXAMPLES:

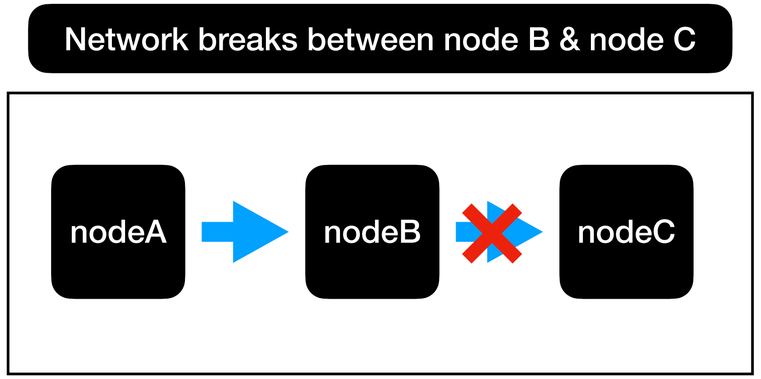
- Most SaaS products target: 99.9% (three nines)
- Cloud providers (AWS, GCP): 99.99% for compute
- Banking/Financial: 99.999% (Five nines)
- Air Traffic Control: 99.9999% (six nines)



Partition Tolerance

- The system continues to operate even when parts of the cluster cannot communicate with each other.

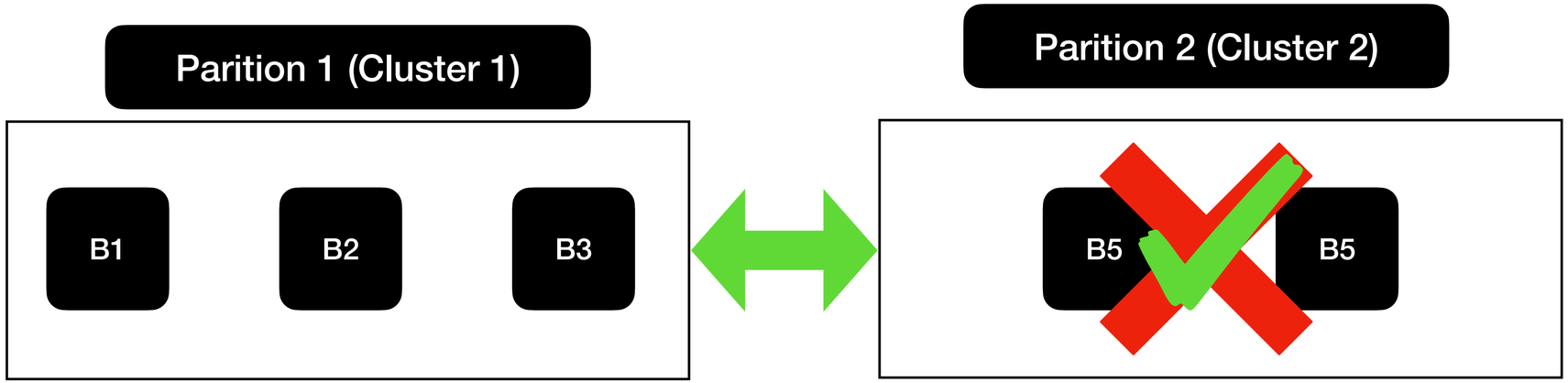




Common causes:

- Firewall misconfigurotion
- Switch/router fallure
  Undersea cable cut
  Cloud proulder network issues
- DDOS atlocks on nefwork infrastfucture

How Kafka handles Parition Tolerance ?



- Kafka has a QUORUM rule:
  - Majority (N/2 + 1) must agree
  - For 5 brokers : 5/2 +1 = 3
- Only Partition 1 can accept writes
- Cluster 2 heals, simply rejoins and syncs up.

SetUp
Kafka Cluster
Using
Three Brokers

Why Not 1 or 2 Brokers?

- 1 Broker
  - No replication
  - Single point of failure
  - Cannot demonstrate distributed behavior
- 2 Brokers
  - Replication possible, but no true majority
  - 1 failure = no quorum (1/2 is not > 50%)
  - Hard to demonstrate real fault tolerance

Why 3 Brokers is the Sweet Spot ?

- 3 Brokers = Smallest Real Distributed Setup
  - Majority = 2 (true quorum)
  - Can lose 1 broker and continue
  - Replication Factor = 3 (industry standard)
  - Enables ISR demonstration
  - Makes min.insync.replicas=2 meaningful
  - Perfect balance: small for laptop, large for concepts
- **Key Idea:**
  - 3 brokers is the smallest cluster that behaves like production.